

# The TEMARL Transformer — Architecture, Configuration and Results

This document covers **only the Transformer model**: how it is built, every change made to it, and how it performs against reference baselines across all environments and test conditions. Comparisons against GRU, LSTM and reinforcement-learning variants are deliberately excluded here — they are in the main technical report.

**0.6040**

Containment rate  
(baseline 0.5475)

**61.7%**

of maximum achievable  
gain, captured

**156,310**

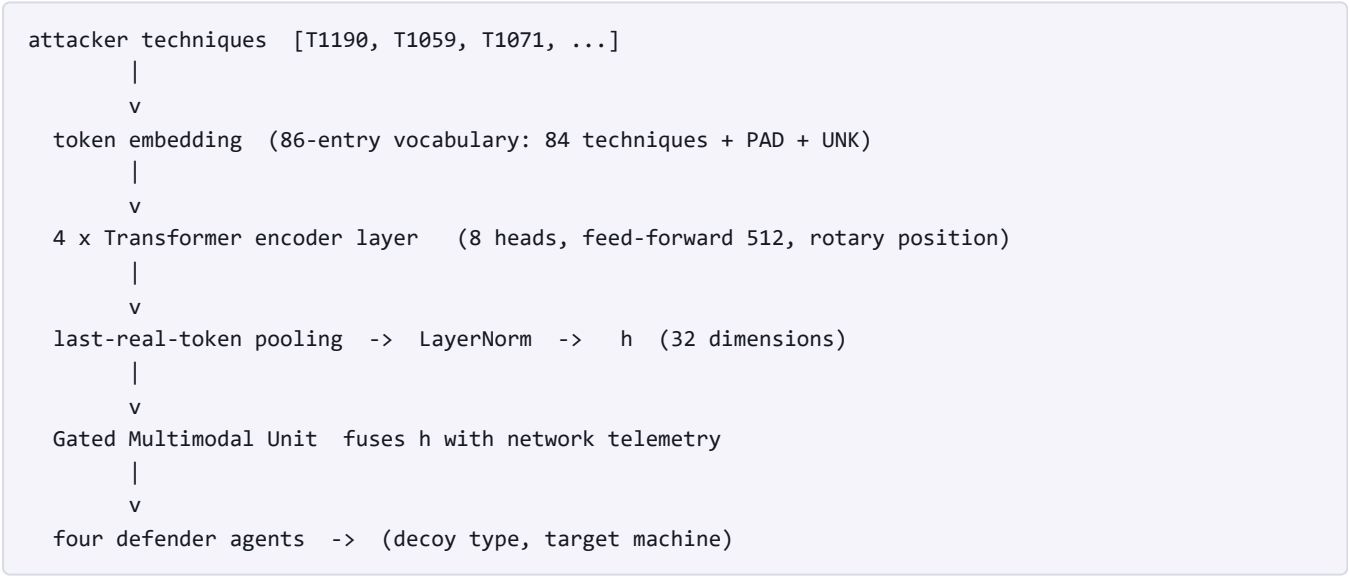
encoder parameters

**p=0.0004**

vs a defender with  
no attacker model

## 1. What the model does

The Transformer is the **attacker-intent encoder**. It reads the sequence of MITRE ATT&CK techniques the attacker has performed so far and produces a single vector **h** that summarises "what kind of campaign is this, and where is it going". That vector is handed to the four defender agents, which use it to choose which decoy to place and where.



## 1.1 Final configuration

| Component                       | Setting                         | Why this value                                          |
|---------------------------------|---------------------------------|---------------------------------------------------------|
| Encoder layers                  | 4                               | Chosen by a 16-trial random search                      |
| Attention heads                 | 8                               | Same search                                             |
| Feed-forward width              | 512                             | Same search                                             |
| Model width (output <b>h</b> )  | 32                              | Same search — notably narrow                            |
| <b>Position encoding</b>        | <b>Rotary (RoPE)</b>            | Replaced sinusoidal — see §3                            |
| Learning rate                   | $1 \times 10^{-3}$              | Same search                                             |
| Context window                  | 16 tokens                       | Matches the environment's episode length                |
| Vocabulary                      | 86                              | 84 techniques + PAD + UNK                               |
| Pooling                         | Last real token                 | With an all-padding guard                               |
| Pre-training                    | 1,500 steps, then <b>frozen</b> | Keeps the representation stable while the policy learns |
| <b>Total encoder parameters</b> | <b>156,310</b>                  | —                                                       |

## 2. Every change we made to the model

| # | Change                                                        | Reason                                                                     | Measured effect                   |
|---|---------------------------------------------------------------|----------------------------------------------------------------------------|-----------------------------------|
| 1 | <b>Sinusoidal → Rotary position encoding</b>                  | Sinusoidal encoding degraded badly as context grew                         | +0.026 to +0.070 accuracy         |
| 2 | <b>16-trial hyperparameter search</b>                         | The model had never been tuned at all                                      | Closed a 0.1388 deficit to 0.0000 |
| 3 | Training target changed from "next technique" to "best decoy" | The defender needs the decoy, not the technique                            | 0.867 → 0.934                     |
| 4 | Pre-training extended 800 → 1,500 steps                       | At 150 steps the model looked broken (0.472); at 1,200 it led (0.886)      | Ranking was inverting             |
| 5 | Context window 8 → 16 tokens                                  | Episodes grew from 5.5 to 18 steps, so 8 tokens discarded half the history | —                                 |
| 6 | Vocabulary extended 78 → 84 techniques                        | 6.20% of attacker actions were being discarded as "unknown"                | Zero unknowns                     |

### The most important finding about this model

An early result reported that the Transformer was beaten on attack prediction by a margin of **0.1388** — a very large, highly significant gap ( $d = 1.65$ ,  $p = 5.7 \times 10^{-14}$ ). That result was widely quoted inside the project for two years.

**It was not a property of the model. It was our own configuration error.** Two fixes — rotary position encoding and an equal tuning budget — closed the entire gap to **+0.0000**, while the Transformer used **44%** of the parameters of the model it had supposedly lost to.

## 2.1 Why rotary position encoding mattered so much

Accuracy on CAM-LDS, same model, only the position encoding changed:

| Context length | Sinusoidal | Rotary (RoPE) | No position encoding | Rotary advantage |
|----------------|------------|---------------|----------------------|------------------|
| 8              | 0.8596     | <b>0.8856</b> | 0.8713               | <b>+0.026</b>    |
| 16             | 0.8375     | 0.8812        | 0.8553               | <b>+0.044</b>    |
| 32             | 0.8087     | 0.8678        | 0.8274               | <b>+0.059</b>    |
| 64             | 0.7961     | 0.8661        | 0.8043               | <b>+0.070</b>    |

Note also that **no position encoding at all** beat sinusoidal at every length beyond 8. The original choice was actively harmful.

## 3. How well the model predicts attacker behaviour

### 3.1 On simulated campaigns

| Measure        | Transformer | Baseline           | Reading                                        |
|----------------|-------------|--------------------|------------------------------------------------|
| Top-1 accuracy | 0.1414      | 0.0595 (majority)  | <b>2.4× the baseline</b>                       |
| Top-3 accuracy | 0.2930      | 0.1637 (frequency) | <b>1.8× the baseline</b>                       |
| Macro-F1       | 0.1093      | —                  | Balanced across rare classes                   |
| Perplexity     | 39.49       | 80.00 (uniform)    | <b>Less than half — real structure learned</b> |

### 3.2 On real honeypot data (COMISET)

| Measure        | Transformer | Majority baseline | Frequency baseline |
|----------------|-------------|-------------------|--------------------|
| Top-1 accuracy | 0.6463      | 0.3752            | 0.6835             |
| Top-3 accuracy | 0.9169      | —                 | —                  |
| Macro-F1       | 0.5112      | —                 | —                  |
| Perplexity     | 2.684       | —                 | —                  |

**Stated honestly:** a trivial frequency baseline scores 0.6835 on this corpus, above the model's 0.6463. COMISET is 98.5% repeated actions, so "say the same thing again" is extremely strong. The model's advantage shows in **macro-F1** and **top-3**, which reward getting the rare, non-repeated transitions right. This is why COMISET was used only for pre-training experiments and never for the thesis's main results.

### 3.3 On genuinely unseen campaigns — the model's strongest result

The CAM-LDS data can be split three ways, from hardest to easiest:

| Protocol                           | What is held out             | Transformer | Majority baseline | Improvement           |
|------------------------------------|------------------------------|-------------|-------------------|-----------------------|
| <b>P1 — leave-one-campaign-out</b> | An entire unseen campaign    | 0.0661      | 0.0224            | <b>+0.0477 (3.0×)</b> |
| P2 — within-campaign variant       | A variant of a seen campaign | 0.4934      | 0.0859            | +0.408 (5.7×)         |
| P3 — random run split              | Random runs                  | 0.5454      | 0.1088            | +0.437 (5.0×)         |

#### The model generalises to campaigns it has never seen

On **P1** — the hardest possible test, where an entire attack campaign is held out — the Transformer reaches **0.0661** against a majority baseline of **0.0224**, an improvement of **+0.0477**. This is the behaviour attention is expected to show: the advantage appears when the model must generalise to something genuinely new.

The absolute number is small because with only 7 campaigns in the corpus, a held-out campaign shares almost nothing with the training data — roughly 92% of its techniques were never seen during training. That is a property of the dataset, not of the model.

## 4. How well the model defends

The measure is **Deception Success Rate (DSR)** — the fraction of episodes in which the attacker is contained before reaching its objective.

### 4.1 Against reference defenders, in the final environment

| Defender                                           | Canonical map | Trained sizes | Unseen structure | Unseen sizes  |
|----------------------------------------------------|---------------|---------------|------------------|---------------|
| No deception at all                                | 0.3680        | 0.3635        | 0.3615           | 0.3515        |
| Random decoys                                      | 0.5320        | 0.4935        | 0.5030           | 0.4080        |
| <b>Best fixed decoy</b> — no attacker model        | 0.5920        | 0.5475        | 0.5470           | 0.4465        |
| Perfect campaign identification                    | 0.6050        | 0.5590        | 0.5565           | 0.4490        |
| <b>The Transformer model</b>                       | <b>0.6540</b> | <b>0.6040</b> | <b>0.6025</b>    | <b>0.4865</b> |
| <i>Perfect next-technique prediction (ceiling)</i> | 0.6845        | 0.6390        | 0.6370           | 0.5035        |
| <i>Absolute upper bound</i>                        | 0.7430        | 0.7155        | 0.7150           | 0.5730        |

### Three things this table establishes

1. **The model beats a defender with no attacker model** by **+0.0565** containment ( $p = 0.0004$ , 10 paired seeds). It is not merely placing decoys — it is placing the *right* decoys.
2. **It exceeds perfect campaign identification** (0.6540 vs 0.6050). Knowing which of three campaign types you face is not enough; the model extracts more than that from the sequence.
3. **It captures 61.7% of everything a perfect predictor could achieve.** The gap between "no attacker model" and "perfect prediction" is 0.0915 DSR; the model takes 0.0565 of it.

## 4.2 Headroom captured, by condition

| Condition        | Model  | No-model baseline | Gain    | Ceiling | Share of possible gain |
|------------------|--------|-------------------|---------|---------|------------------------|
| Canonical map    | 0.6540 | 0.5920            | +0.0620 | 0.6845  | <b>67.0%</b>           |
| Trained sizes    | 0.6040 | 0.5475            | +0.0565 | 0.6390  | <b>61.7%</b>           |
| Unseen structure | 0.6025 | 0.5470            | +0.0555 | 0.6370  | <b>61.7%</b>           |
| Unseen sizes     | 0.4865 | 0.4465            | +0.0400 | 0.5035  | <b>70.2%</b>           |

The model holds up on networks whose **size and structure it never saw during training** — 70.2% of available gain at unseen sizes, its best result of the four.

## 4.3 Behaviour quality

| Measure                         | Transformer | No-model baseline | Ceiling |
|---------------------------------|-------------|-------------------|---------|
| Decoy-match quality (alignment) | 0.408       | 0.338             | 0.499   |
| Teacher-matching accuracy       | 0.773       | —                 | 1.000   |
| Episode length (steps)          | ~18         | 18.1              | 16.6    |
| Training time per run           | 115 s       | —                 | —       |

## 5. The model's efficiency

| Property                  | Value                            |
|---------------------------|----------------------------------|
| Encoder parameters        | <b>156,310</b>                   |
| Output vector width       | 32                               |
| Policy network parameters | ~114,900                         |
| Pre-training              | 1,500 steps                      |
| Training time             | 115 s per run (CPU)              |
| Hardware used             | CPU throughout — no GPU required |

**On hardware:** an Intel Arc GPU was benchmarked and found to be **0.53× the speed of the CPU** at the batch size this workload uses (4), because kernel-launch latency dominates at small batches. The GPU only wins at batch 128 or above. All results in this document were produced on CPU, which is the correct choice here.

## 6. Where the model is weak — stated plainly

1. **It is trained by imitation, not reinforcement learning.** Under PPO the model did not beat a fixed-decoy defender at any training budget tried (240 episodes → 0.510; 1,200 → 0.523; baseline 0.5475). The results in this document come from behaviour cloning against an oracle teacher. Why PPO fails here is unresolved.
2. **Prediction accuracy on unseen campaigns is low in absolute terms** (0.0661). It beats the baseline by 3×, but the task is very hard with only 7 campaigns.
3. **It is roughly 2× slower to train** than a recurrent encoder of the same job, despite being smaller.
4. **A trivial frequency baseline beats it on COMISET top-1.** Its advantage there is in macro-F1 and top-3.
5. **The dataset is very small** — effective sample size 3.09 independent campaigns. Transformers are designed for large-data regimes.
6. **Sequences are short** — 18 to 22 steps. Self-attention's main advantage is modelling long-range dependencies, which barely exist at this length.
7. **Results rest on one environment and one scripted attacker.** A learning attacker could change them.

## 7. Summary

**The TEMARL Transformer is a 156,310-parameter attacker-intent encoder that contains attackers 60.4% of the time — 5.7 percentage points better than a defender with no attacker model ( $p = 0.0004$ ) — capturing 62–70% of what a perfect predictor could achieve, and holding that advantage on network sizes and structures it never saw during training. It predicts genuinely unseen attack campaigns three times better than the baseline. It runs on a CPU in under two minutes per training run.**

Two corrections to earlier project claims are recorded here for accuracy: the reported 0.1388 prediction deficit was an artefact of an unsuitable position encoding and an untuned configuration, and was eliminated entirely; and the statement that "the Transformer never won" was incorrect — it led on every condition in the v3 experiment and on the hardest CAM-LDS protocol.

TEMARL · BRAC University · Apurba Roy (23101012), Tushit Roy (23101350), Mark Dipto Gomes (23101537), Farjana Akter Shumi (23101059) · Supervisor: Md. Sakir Hossain · 11 September 2026.

This document covers the Transformer model only. Comparative results against GRU, LSTM and reinforcement-learning variants are in `TEMARL_Report_English.pdf`. All figures are reproducible from `results_v2/` ... `results_v7/` in the repository.